GFX2D：基于 RISC-V 与 FPGA 异构架构的 2D 图形渲染加速引擎

摘　要

本作品在 Ti60F225 开发板上自研了由 RISC-V 软核调用的 2D 硬件绘图引擎，并配套 C 驱动、软件渲染器和可扩展的游戏机前端，画面经 HDMI 实时输出，构成图形加速系统 GFX2D。处理器每帧更新状态并提交绘图命令，硬件完成图像块拷贝、纯色填充、Alpha 半透明混合与色键去背，结果写入 DDR3 帧缓冲；显示链路经双缓冲输出 640×480@60 Hz 画面。普通游戏短测从纯软件的 2.5 FPS 提升到硬件的 60 FPS，约 24 倍。常见 2D 加速引擎逐块直写帧缓冲，同屏活动元素（Sprite）增多，帧率随之下降；赛题要求开启混合与色键后保持 60 FPS，真正的考验是同屏活动元素的数量极限。为此引擎借鉴主流 GPU 的纹理缓存与分块渲染，形成三项创新：高频素材预先装入片上纹理，绘制时不再读 DDR3；条带缓冲把同一区域内的多次像素合成留在片上再写回帧缓冲，代价是 CPU 要替每个条带挑出 Sprite 各存一份命令；硬件分桶把这步交给 FPGA，只存 8 字节引用，不再复制命令。同一比特流、同一程序对照中，8×8 Sprite 的稳定 60 FPS 极限 N_{max} 从 1947 提升到 2187，增幅 12.3%；每帧 DDR3 读、写突发分别减少约 70% 和 92%。条带缓冲模式进一步扩展到三维：增加硬件三角形光栅与深度缓冲，板上实时运行可交互的三维走廊轨道射击（透视门框、敌机与弹幕），连续 1800 帧保持 30 FPS。整机占用 60,064 XLR、172 块 RAM、23 个 DSP。**系统完成赛题 3 项基础要求与 3 项高阶要求，片上纹理、条带缓冲、硬件分桶与三维扩展构成四项创新成果。**

关键词：2D 图形加速；FPGA；RISC-V；软硬件协同；DDR3

表 1　赛题要求完成情况概览（详细结果见 3.3.5 节）

类别	赛题要求	完成情况
基础要求	解析渲染指令的操作、地址与宽高	✅ 已完成，硬件解析操作、源地址、目的地址、宽、高，逐项校验
	硬件引擎支持图像块拷贝与纯色填充	✅ 已完成，硬件执行块拷贝与纯色填充，输出与软件参考逐像素一致
	HDMI 输出不低于 640×480@60	✅ 已完成，帧缓冲经 TMDS 编码输出 640×480@60 Hz 画面
	经总线直接访问内存，使用突发与 FIFO	✅ 已完成，引擎作 AXI4 主机以 128 位突发直访 DDR3，配 FIFO 缓冲
	RISC-V 端以 C 语言开发驱动程序	✅ 已完成，C 驱动经寄存器提交命令表并处理中断与错误码
	实现双缓冲机制避免画面撕裂	✅ 已完成，引擎画后缓冲，场消隐期换页，画面无撕裂
	屏幕实时对比软硬件渲染帧率	✅ 已完成，同一画面 HUD 显示软件 2.5 与硬件 60 FPS
高阶要求	利用 DSP 硬件乘法器执行透明度混合	✅ 已完成，12 个 DSP 核并行完成 4 像素×RGB 的透明混合
	颜色键控去背，跳过透明色像素	✅ 已完成，源像素与色键比较，命中即跳过，异形轮廓无残留
	开启混合与色键保持 60 FPS 挑战极限	✅ 已完成，8×8 稳定 60 FPS 达 2187 颗，3600 场零掉帧
创新成果	高频小素材装载后驻留片上存储	✅ 已实现，高频素材驻留 16 KiB 片上纹理，源读取不再访问 DDR3
	目的区域在片上合成后整带连续写回	✅ 已实现，画面分 30 个 640×16 条带，合成后整带连续写回
	硬件生成条带引用表，替代 CPU 复制	✅ 已实现，硬件生成引用表替代 CPU 复制，读/写突发降 70%/92%
	条带框架扩展出三角形光栅与深度缓冲	✅ 已实现，硬件光栅与深度缓冲驱动 3D 走廊，1800 帧 30 FPS

第一部分　作品概述

1.1 功能与特性

本作品实现了一套能在 FPGA 开发板上运行弹幕游戏、直接向 HDMI 显示器输出画面的 2D 图形加速系统。自研部分包括硬件绘图引擎、显示链路、C 驱动、软件执行器和游戏机前端，覆盖赛题基础功能与高负载挑战 [1]。

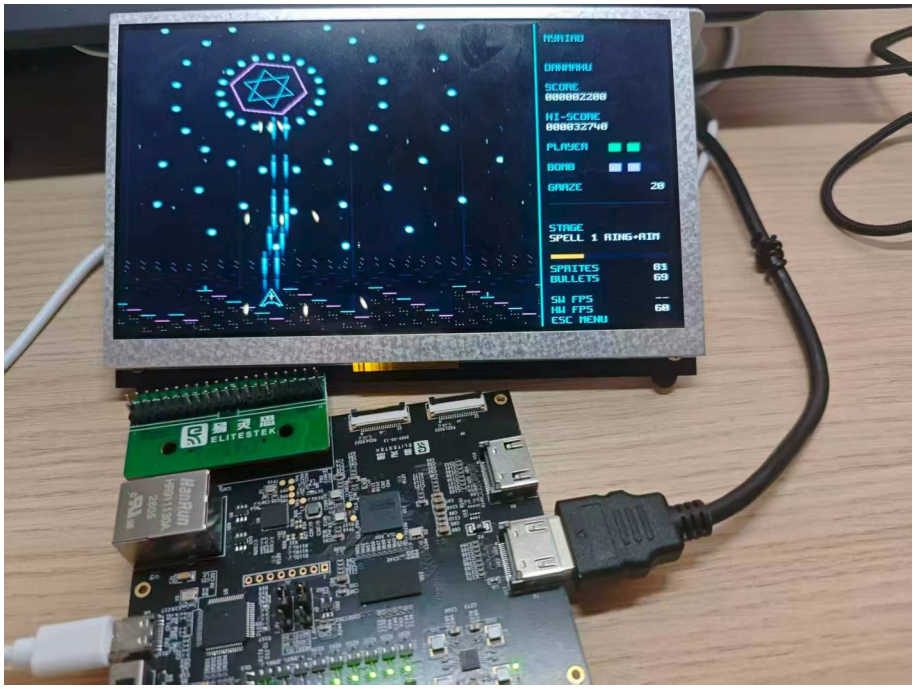


图 1　本作品运行实拍

RISC-V 处理器承担输入、运动与碰撞计算，并将本帧所需的背景、角色与弹幕整理为绘制任务；FPGA 引擎据此完成贴图搬运、色块填充、半透明叠加与按指定颜色去除背景，结果写入帧缓冲后由显示通路输出至 HDMI。双缓冲使绘制与显示各占用一块缓冲，避免画面撕裂。针对同屏小图块数量大、复用率高且相互重叠的特点，引擎将常用贴图驻留于芯片内部存储反复取用，按屏幕横条就地合成后整条写回，各横条所需的绘制内容亦由硬件统计与排序。前端提供基础演示、弹幕游戏、压力测试与三维走廊轨道射击四个应用，可在菜单间切换并实时显示帧率与绘制数量；在半透明与去背开启的条件下，同屏 2187 颗 8×8 弹幕仍维持 60 FPS。

1.2 应用领域

从早期 2D 游戏机到当代嵌入式图形界面，画面的生成长期依赖处理器逐像素搬运与混合。这类重复操作占用了大量处理器时间，帧率随图层与绘制元素增加而下降。图形界面的共性在于素材高度复用、图层相互遮盖，绘制数量又随状态起伏；通用处理器并不擅长此类逐像素工作，异构计算与软硬件协同因而成为主流做法。



图 2　医疗与车载界面 [9]

本系统适用于工业控制、医疗监护、车载仪表、智能家居与游戏娱乐等二维界面。这些界面的绘制任务以搬运、填充和图层叠加为主，对稳定的帧率与较低的主控成本有共同诉求。由专用硬件承担像素级操作、处理器专注于业务逻辑，可在不改变上层界面结构的前提下提升整体表现。

1.3 主要技术特点

系统以命令驱动方式组织绘制：处理器只更新状态并提交一组绘图任务，之后由引擎自主取指、访问内存并完成像素运算。基础流水线在连续供数时每周期处理 4 个像素，4 个像素的 RGB 三通道由 12 个 DSP 并行完成混合。

针对源读取、目的写回与命令构造的三项手段，最终收敛为直通、条带和分桶三种执行路径。三者共用同一套二维操作语义，同一应用可在其间切换，使不同实现与软硬件方案能在相同负载下对照。

软件在 PC 与板上双目标构建，画面先在 PC 上验证再上板运行。硬件按目标负载分为二维与三维两种构型，分别在逻辑与存储资源之间取舍。

1.4 主要性能指标

表 2　主要性能指标（半透明与色键开启）

指标	结果
显示输出	640×480@60 Hz
普通游戏短测	约 2.5→60 FPS，约 24 倍
同屏活动元素极限	约 2200 颗，稳定 60 FPS
等效混合吞吐	整机 33.6、峰值 400（Mpix/s）
三维场景	30 FPS，每帧两场
整机资源	60,064 XLR（98.8%）/ 172 块 RAM / 23 DSP

1.5 主要创新点

借鉴主流 GPU 的纹理缓存与分块渲染，针对弹幕负载形成四项实现：

（1）片上纹理：高频素材装载后复用，减少 DDR 源读取。

（2）条带缓冲：同一区域片上合成，减少零散访问并连续写回。

（3）硬件分桶：FPGA 生成保序引用，替代 CPU 分桶与命令复制。

（4）三维扩展：条带框架加三角形光栅与深度缓冲。

1.6 设计流程

设计流程由赛题要求与游戏负载共同确定软硬件分工。首先定义命令接口与表面登记方式，完成基础绘图、双缓冲和显示链路，使分工可运行、可对照；随后按帧分段测量，把一帧的时间拆到源读取、目的写回与命令构造三处，据此依次优化，每一步都用同一负载做前后对照；最后扩展三维通路，二维与三维两种构型分别综合、上板测量，并回归既有功能。

第二部分　系统组成及功能说明

2.1 整体介绍

图 3 给出系统总体架构。Ti60F225 内部由三部分组成：运行 C 程序的 RISC-V 软核、自研的 GFX2D 二维绘图引擎，以及把帧缓冲转换为 HDMI 时序的扫描输出；外部 DDR3 保存程序、素材、命令和帧缓冲，三者共用一条 128 位总线访问它。

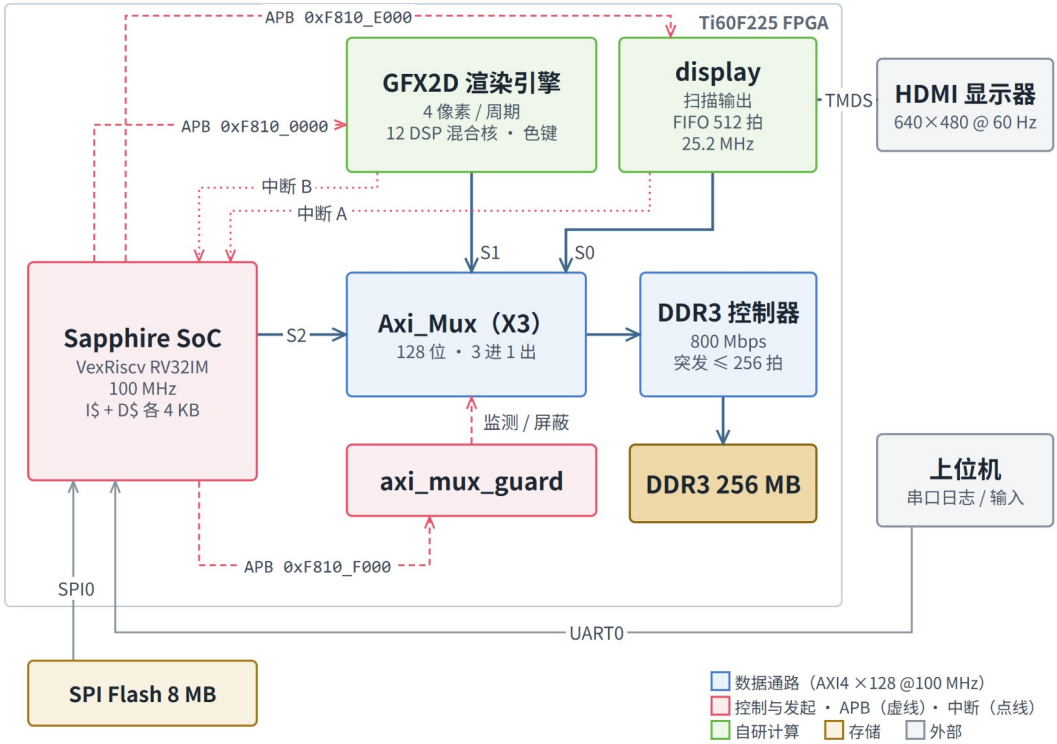


图 3　系统总体架构

软核、DDR 控制器和总线仲裁采用厂商 IP，绘图引擎、扫描链路与配套 C 软件为自研部分。APB 是控制通路，驱动经它配置寄存器；AXI4 是数据通路，承担像素与命令的批量传输。引擎完成一次任务或检测到错误时以中断通知处理器，软件不必逐拍干预搬运。

以绘制一颗移动弹幕为例。处理器更新它的坐标，生成一条包含素材、目标位置和混合方式的命令；引擎读取该素材，把有效像素与背景混合后写入后帧缓冲；本帧对象全部完成后，软件请求换页，扫描输出在下一场消隐期将新帧切换为前台。整屏按固定顺序合成，背景、弹幕、角色与 HUD 依次叠加。

处理器与引擎各用两份命令表交替工作：引擎执行上一帧时，处理器准备下一帧，正在执行的表保持不变；双帧缓冲则隔开绘制与扫描。2.2 先说明基础通路，再给出源读取、目的写回与命令构造三方面的改进，2.3 说明软件结构。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

作品使用赛题指定的 Ti60F225I3-V4 开发板，板载 256 MB DDR3、8 MB SPI Flash 和 HDMI 接口 [3]。图 4 给出 DDR3 与 SPI Flash 的分区。程序、素材与离屏层各有独立区域，其中离屏层为不直接送显的绘制缓冲；双命令表与双帧缓冲各占两块区域交替使用，Flash 保存比特流、应用与素材的发布分区。

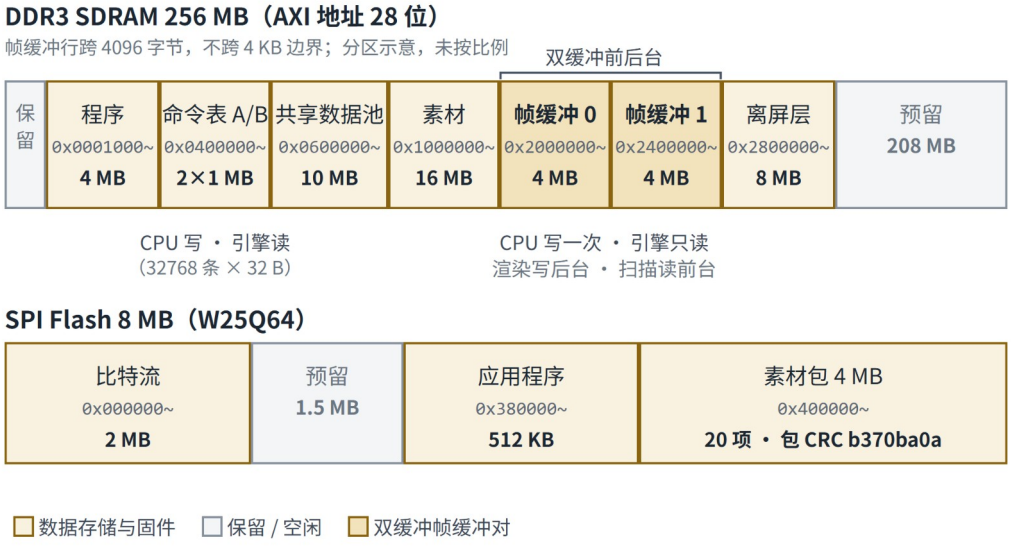


图 4　DDR3 与 SPI Flash 的基础地址分区

核心与总线工作于 100 MHz，显示像素时钟为 25.2 MHz，两者相互独立，绘图与扫描因而可以并行。

帧缓冲采用 XRGB8888，RGB 各占 8 位，高字节不作为透明度；纹理采用预乘 ARGB8888，额外保存 Alpha。两种格式均以 32 位存储，一个 128 位总线拍正好承载 4 个像素，使取数、混合和写回能够按同一宽度衔接。纹理在打包阶段预乘透明度，相关运算见基础像素通路。

一帧 640×480 的 32 位图像约需 9.8 Mbit（1.2 MB），而片上 RAM 共 256 块、每块 10 kbit，合计 2.56 Mbit，容不下整帧。因此整帧、完整素材和命令表放在 DDR3，片上存储只用于当前反复访问的小范围数据：高频纹理、正在合成的条带和跨时钟 FIFO。有限的片上容量由此服务于数据复用，大容量 DDR3 保存完整场景。

2.2.2 电路各模块介绍

2.2.2.1 命令与几何模块

基础引擎的输入是一份绘图命令表，输出是后帧缓冲中的像素。图 5 将其划分为寄存器与命令取数、检查与解码、几何处理、像素处理和 AXI 事务层五部分。

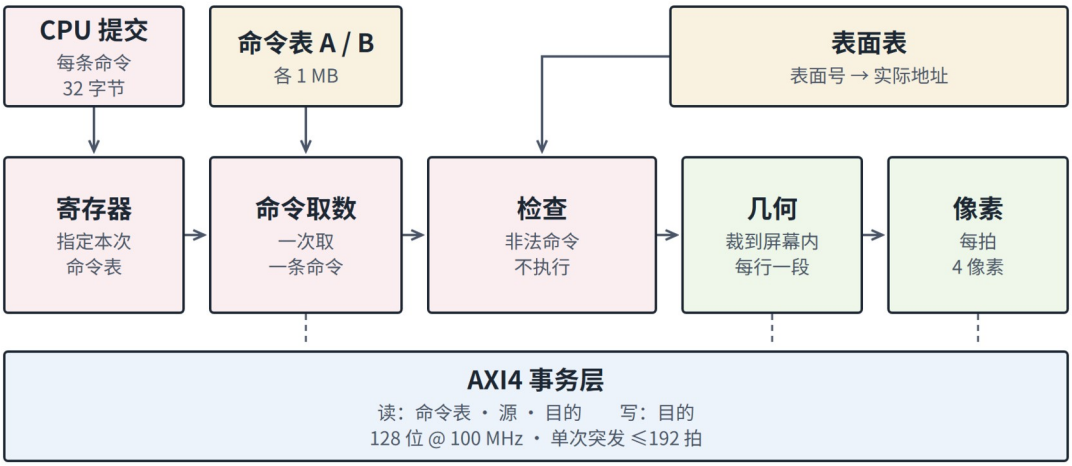


图 5　基础引擎的命令流程

CPU 在提交前配置表面表，登记每张图像的基址、宽高、行跨和像素格式；命令只引用表面号与矩形坐标，硬件据此换算实际地址，同一套操作因而

可用于纹理、帧缓冲与离屏图层。每条命令固定 32 字节，共五种：纯色填充、图像块拷贝、设置裁剪区域、空操作和结束。

驱动写入命令后设置命令表起始地址与条数、再写启动位；引擎自主取指，结束命令与设定条数先到者为准。前端先检查操作与表面是否合法，几何层求有效范围并把矩形展开为逐行数据：移出屏幕或被裁剪的部分不再生成像素访问，保留部分的源、目的坐标同步调整；AXI 事务层按行段搬运，使像素流水线只接收实际参与绘制的内容。

字段或表面非法时该命令在写入前被拒绝；总线错误与超时按在途事务排空后上报错误码与索引。

2.2.2.2 像素混合模块

图 6 的像素通路每次接收一个 128 位拍，即 4 个像素。每个像素的红、绿、蓝三个通道独立计算，因而例化 4×3 个混合核，共使用 12 个 DSP。流水接收源与目的数据，进行色键比较、乘法、加法和舍入，再输出写回像素及有效掩码。连续供数且没有反压时可每周期处理 4 个像素；实际帧率还受取数、命令准备和显示同步影响。

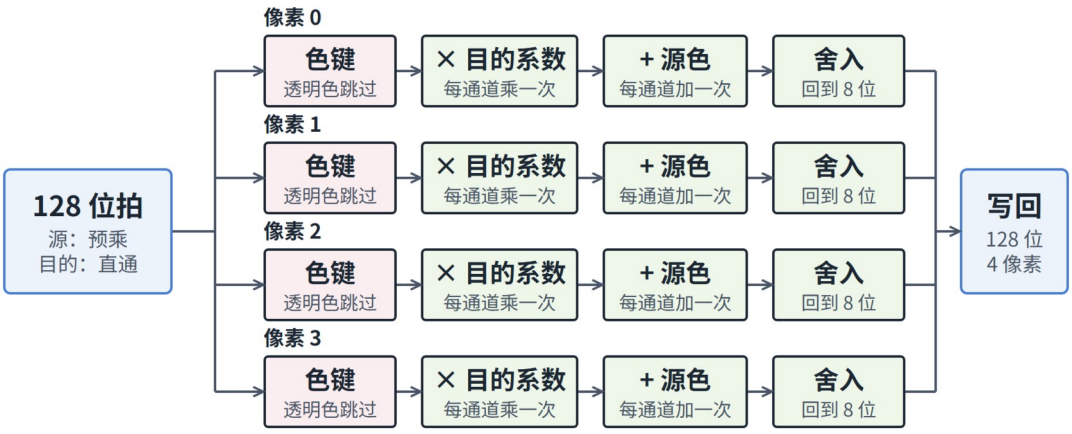


图 6　像素流水线与混合方程

混合按「输出 = 源 × 源系数 + 目的 × 目的系数」计算，系数以 255 为满量程归一化；Alpha 合成语义参考 Porter–Duff 方法 [4]。六种方程共用这条通

路，包括直接复制、半透明合成、发光叠加与暗化等。源颜色已预乘 Alpha，主路径每通道只需再算目的颜色的乘法，结果舍入并饱和至 8 位。

色键解决素材矩形之外的透明背景。图 7 对比色键关闭与开启的结果。引擎将源存储字与指定透明色比较，命中时关闭该像素的写使能，原有目的像素保持不变；未命中时按所选混合方程计算。由此可显示带透明边缘的弹幕。

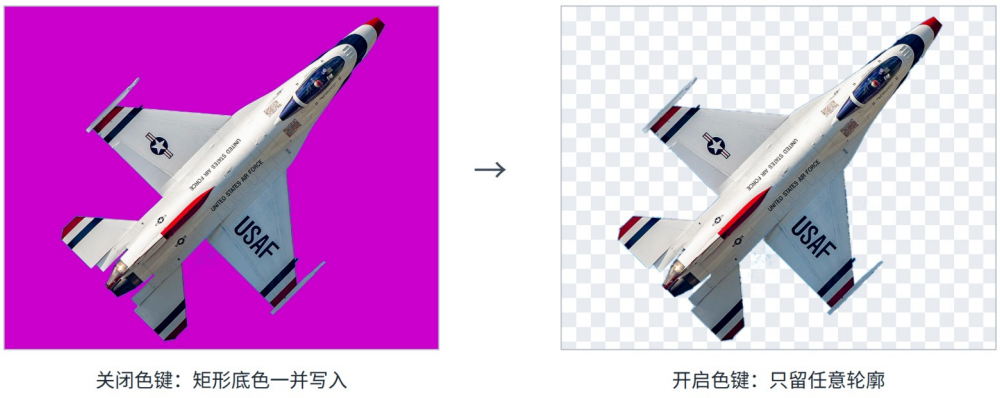


图 7　色键关闭与开启

2.2.2.3 扫描输出与双缓冲模块

显示通路独立于绘图执行，见图 8。取数模块按扫描顺序从前帧缓冲逐行读取，一行 640 像素对应 160 个总线拍，在 100 MHz 下约 1.6 µs，占一行时间的 6.3%；128 位×512 深异步 FIFO 把数据交给 25.2 MHz 像素域，既隔开时钟，也吸收访存等待。输出模块每拍拆出 4 个像素，经 TMDS 编码以 640×480@60 Hz 输出；一场 480 行约 12.2 ms，加消隐后 16.7 ms。

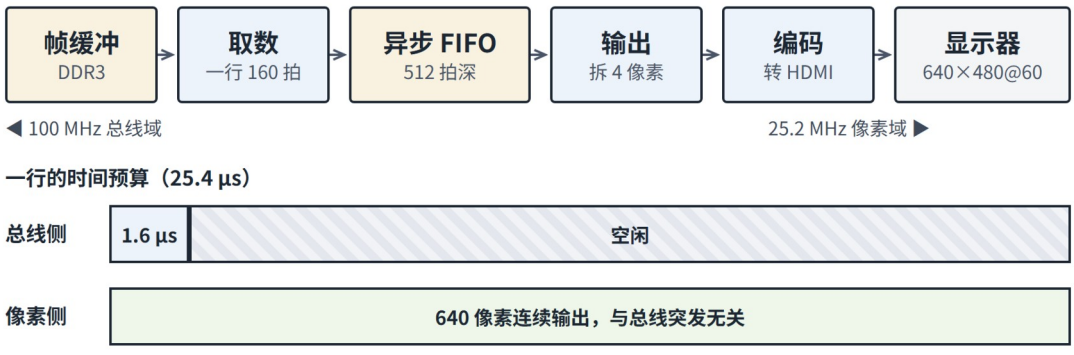


图 8　扫描通路与一行时间预算

双缓冲流程见图 9。显示始终读前缓冲，引擎在后缓冲绘制；软件确认本帧完成后写入后缓冲起始地址并置换页请求，硬件在下一场消隐期锁存新地址。若本帧尚未画完，则继续显示上一幅完整画面，不把绘制到一半的内容交给扫描。交换只在消隐期发生，绘制与扫描因而不会同时访问同一块缓冲。

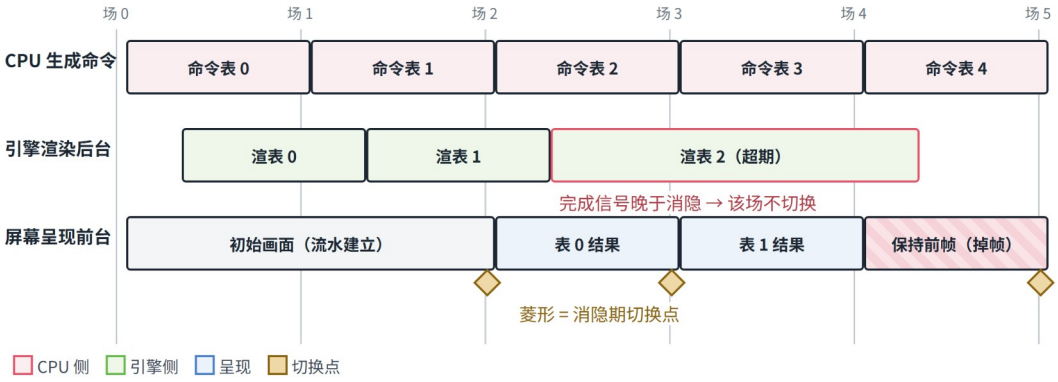


图 9　双缓冲与命令表交替

2.2.2.4 片上缓存与分桶调度模块

基础流水线能并行计算像素，但一帧还要为大量小 Sprite 读取源图、访问目的像素并准备命令：弹幕常复用同一张图片却分布在不同位置，重叠对象又反复修改相同区域。引擎借鉴图块化渲染利用空间局部性的思路 [5]，分别优化源数据、目的数据和命令组织，三步都保持原有绘制顺序与像素语义。

（1）片上纹理：复用重复读取的小素材。 许多弹幕的区别只是坐标，同一种弹型仍用同一张源图；若每绘制一颗都从 DDR3 读取小块素材，源数据虽少，事务固定开销仍会反复出现。本作品的圆弹、火花等高频热集约为 10 KiB，可放入 16 KiB 片上纹理存储。

应用在关卡装载阶段经 APB 写入热素材，此后相同素材直接从片上读；存储按四 bank 组织，搬运层按源标记选择片上端口或 DDR，裁剪与混合方式不变。热集由软件选择与装载，大图与其他素材仍使用 DDR。约 27 周期的读事务固定成本不再施加于这些热素材访问，目的帧缓冲仍需按绘制任务读改写。

（2）条带缓冲：集中目的区域的像素合成。 为减少目的区域反复进出 DDR，将 640×480 画面划为 30 个 640×16 条带。图 10 对比直通与条带的写回次数。引擎一次处理一个条带，执行 LOAD、EXEC、FLUSH 三个阶段：先装入背景，再在片上缓冲执行覆盖该区域的命令，最后把完整条带连续写回帧缓冲。

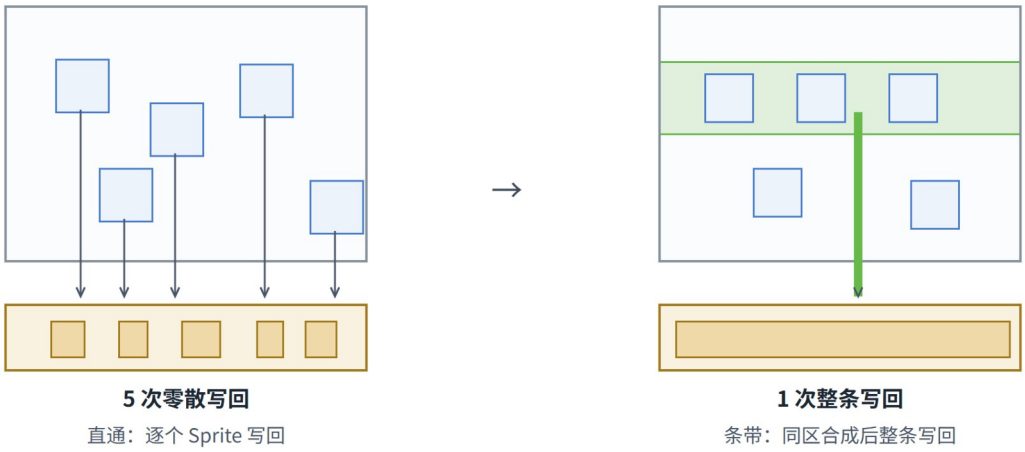


图 10　直通与条带的写回次数

同一区域内的重叠弹幕共享片上目的数据，多次混合在缓冲中完成。跨条带的 Sprite 分别进入覆盖的桶，换算为局部坐标并按交集裁剪；各桶保持原顺序，半透明叠加结果与逐对象直通绘制一致。片上只保留当前条带，完成后复用同一块存储处理下一条带。

该组织增加 CPU 工作：每个对象都要计算覆盖条带、换算局部坐标并向多个桶复制命令。板上测得 CPU 逻辑与组帧合计约 13 ms、引擎执行约 6 ms，整机 N_{max} 低于同期直通路径；访存已改善，瓶颈转到 CPU 一侧，下一步把这些分桶与复制交给硬件。

（3）硬件分桶：由引擎建立紧凑任务引用。 CPU 每帧只发布原始绘制及实例数组，保留纹理、位置与混合状态等必要信息。硬件第一遍完成检查、裁剪和各条带计数，通过前缀和确定引用范围；第二遍按输入顺序把引用散布到各桶。每个桶由此获得该区域的保序绘制子序列。

参数缓冲保存绘制元数据与 8 字节引用；跨桶对象在各桶记录原始数据的身份，执行时按引用取用，免去复制完整命令。保序是半透明合成的前提：后绘制的图层要覆盖或混合到已有结果上，不能按纹理或坐标重排。引用表还携带容量边界，双参数缓冲按帧租借，在执行和排空完成后才可重新使用。

执行阶段沿用条带缓冲与基础像素流水线，改动集中在任务的形成与取用。二维构型保留直通、条带和硬件分桶路径，软件可在同一比特流上切换；片上纹理是可与适用路径组合的源读取机制。各阶段结果见 3.3 节，硬件分桶以同程序对照，历史构型只保留各自的测量范围。三项叠加后，以直通为基准的每帧 DDR 访问量见图 11，降至读 30%、写 8%。

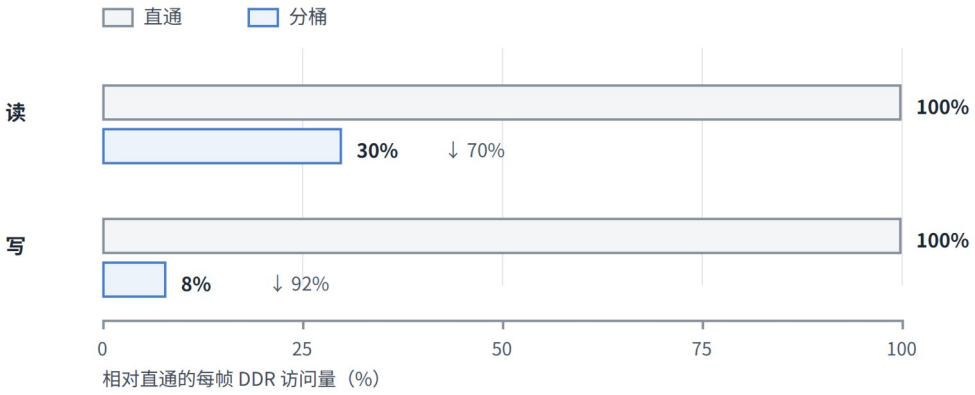


图 11　相对直通的每帧 DDR 访问量

2.2.2.5 三维光栅模块

条带框架按屏幕区域组织工作，每个区域经历装载、执行和写回。三维扩展保留这套组织与调度，把区域内的处理对象换成三角形，并增加光栅取数器、光栅核和深度缓冲；二维通路继续绘制 HUD，复用集中在条带存储、搬运与整帧调度。

CPU 先根据相机和模型变换顶点，完成近远面裁剪与投影，再计算光栅需要的边缘函数及属性增量，写成 128 字节 setup 记录。随后按三角形覆盖范围建立条带索引，使硬件处理某个区域时只读取与之相交的记录。几何准备在 CPU 完成，重复的逐像素覆盖与深度判断交给 FPGA。

光栅核采用 Pineda 边缘函数 [6] 判定像素中心是否位于三角形内部，并用 top-left 规则统一共享边界的归属。图 12 对比关闭与开启深度判断时的遮挡结果，右侧为同一帧的深度值。像素被覆盖后，硬件插值颜色或纹理坐标，完成采样、色键与深度判定；通过的像素更新颜色和深度，前后表面按深度关系遮挡。



图 12　深度判断的作用

运算采用整数定点：几何坐标 Q16.16 的低 16 位为小数，屏幕坐标 Q12.4 保留 4 位；位宽与舍入顺序由同一记录 ABI 规定，CPU 准备的数据可被硬件直接解释。

三维采用独立构型，复用片上纹理与条带缓冲，不启用二维硬件分桶前端；三角形的保序分桶仍由 CPU 生成。两种构型分别选择二维命令组织和三维像素处理所需资源。三维走廊轨道射击的帧时间与资源见 3.3.4 节。

2.2.2.6 总线与存储接口

三项优化所依据的访存成本来自对 DDR3 控制器 [7] 的板上测量。周期模型 $T(N) = t_tx + kN$ 将一笔事务分为固定启动开销与随拍数增加的传输时间。读事务的固定项约为 27.41 周期、每拍增量为 1.086；写事务对应 7.25 和 1.183。短事务反复支付固定成本，长突发可将其分摊，源热集驻留与目的连续写回因而有效。图 13 对比逐拍读取与整行突发的每拍成本。同一行数据若按拍逐次读取，摊到每拍的周期数约为 28.5；改为 160 拍整行突发后降至 1.26。

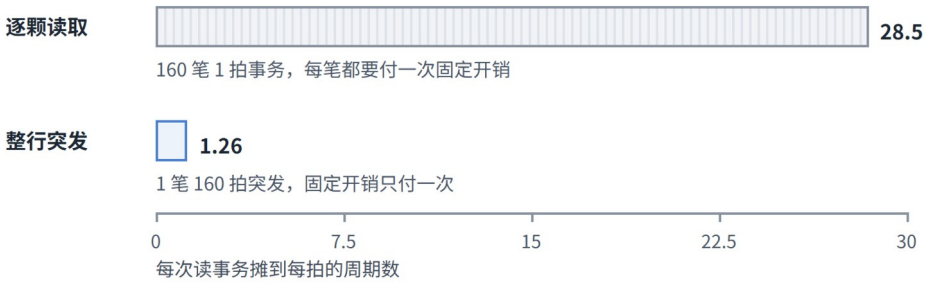


图 13　逐颗读取与整行突发的每拍成本

帧缓冲行跨取 4096 字节。硬件据行号定位行首、按列坐标定位像素，可见行的 160 拍访问落在同一 4 KB 区域，符合 AXI4 突发不跨 4 KB 边界的要求 [8]，也便于显示按行扫描与引擎逐行搬运。

显示接总线端口 0，引擎接端口 1，处理器接端口 2。显示连续实时读取，引擎集中读写，CPU 还要取指与处理状态。显示和引擎限制在途事务数，总线仲裁在处理器地址等待超龄时暂缓引擎的新授权，给 CPU 留出访问机会；显示 FIFO 则吸收短时等待。

CPU 缓存与引擎写入的 DDR 之间没有硬件一致性：驱动发布命令与素材时保持其有效，CPU 读回引擎写入的区域前须先失效缓存；游戏以状态寄存器与中断判定完成，整帧读回只在检查时启用。

2.3 软件系统介绍

2.3.1 软件整体介绍

本作品的软件同时构建两个目标：板上程序运行在 Ti60F225 的 RISC-V 软核上，PC 程序运行在开发机上；两者共用同一份应用与算法代码，只在下层的板级实现上分开。软件自下而上分为四层，关系见图 14：最底层是厂商提供的处理器支持包，其上是板级支持层，再上是算法层，最上层是应用层。

应用层由系统菜单与四个应用构成，决定场景、交互和绘制顺序；算法层把绘制需求转换为引擎命令，并提供按相同语义解释命令的软件执行器；板级支持层封装内存、引擎、显示与输入。上层只调用下层接口，反向不被允许，这条约束由构建时的分层检查保证。

同一个应用可以选择两种后端：硬件后端下，CPU 的输出是命令表或原始绘制数组，由 FPGA 写出帧缓冲像素；软件后端下由 CPU 解释同样的绘制需求并写入像素。两种后端的显示与呈现接口一致，切换只改变绘制任务的执行者，游戏规则与绘制意图不变。

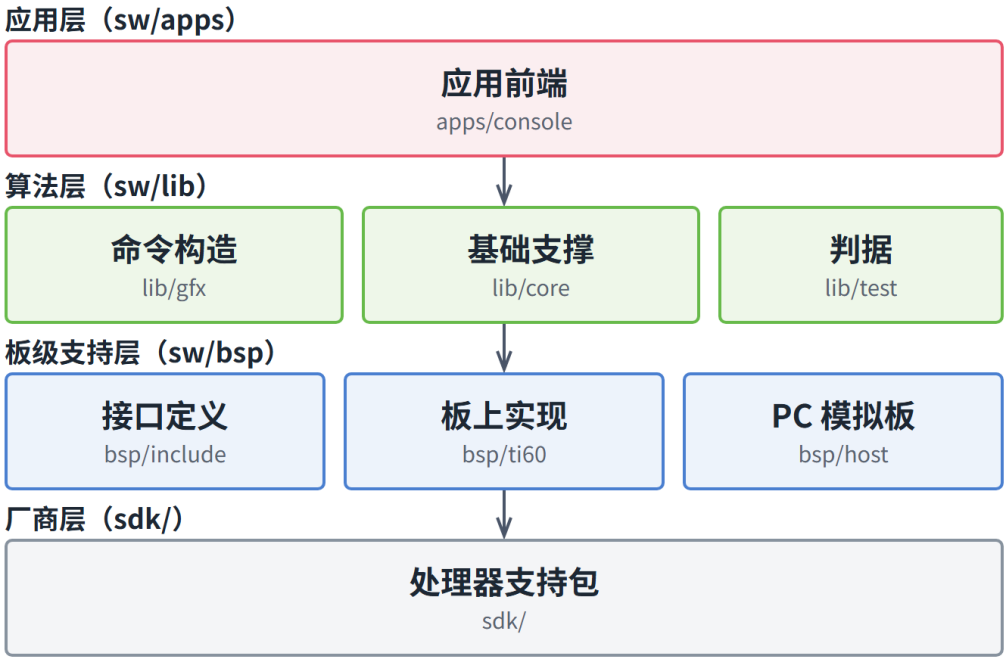


图 14　软件分层与模块划分

2.3.2 软件各模块介绍

2.3.2.1 应用层：框架与四个应用

产品以系统菜单为入口，菜单本身不列入可启动内容。框架按注册表固定应用编号，只枚举可启动项，依次调用进入、逐帧与退出回调；失败时先确认引擎排空，再清理并返回菜单。应用只声明名称与回调。

四个应用各对应一种用途：基础演示、弹幕游戏、压力测试与三维走廊射击，末者使用三维光栅通路。弹幕游戏为竖版射击，左侧 400×480 为游戏区，右侧 240×480 显示分数、残机、活动数量与后端帧率；输入决定角色移动与炸弹，逻辑层更新敌机、弹幕与碰撞，再按顺序生成背景、对象和界面任务。运动采用定点计算，实体池管理敌弹、自弹、粒子与拾取物。

关卡由普通敌机推进到首领战，不同弹幕模式形成圆环、螺旋与狙击等弹型，最终阶段密度可调，用作负载旋钮。每颗活动弹渲染为对应尺寸的 Sprite，同时保留背景、角色、特效与 HUD，数量测量因此包含真实游戏的状态更新与整帧绘制。

演示时由菜单选择应用，应用内再选择渲染后端；HUD 显示场景、后端、帧率与活动数量。输入来自板上按键，或由上位机把键盘与手柄状态经串口转发，上位机不参与板上渲染。

2.3.2.2 算法层：命令构造、执行与呈现

应用提交表面、矩形、坐标与混合状态，命令构造层按所选路径生成可执行任务：直通路径输出 32 字节命令表，条带路径求对象与条带的交集并生成局部命令，分桶路径只发布原始绘制与实例数组。应用接口保持绘制顺序，数据组织的差别由底层承担。

二维串行路径由一个统一入口按后端分派，输入为任务表、表面信息和目标缓冲，输出执行状态与耗时；流水路径分步推进上一帧执行和下一帧准备。驱动写入基址与长度、敲门铃启动，再通过中断或状态判断结果。正在执行的命令与参数缓冲不能提前复用，出错时先处理该任务的收尾。

绘制完成后，串行路径请求目标帧面并等待实际交换，流水路径把这一操作纳入帧调度；应用由此区分“像素已画完”与“画面已显示”，HUD 帧率按实际呈现统计。

三维应用的帧也由算法层构造：更新相机与场景，生成定点三角形记录和条带引用，提交三维光栅通路，二维命令在同一帧中绘制 HUD。该应用按 30 Hz 更新和呈现，一帧对应两个显示场，三类缓冲各自管理生命周期。

2.3.2.3 板级支持层：板上驱动与 PC 模拟

板上侧封装内存与帧缓冲地址、引擎寄存器与中断、显示与输入引脚，应用只调用接口，不直接写寄存器。PC 侧用窗口库模拟同一组接口，以文件代替 Flash 与 DDR3，使同一应用不必修改即可在开发机上运行。

两个目标由同一份构建脚本产生：板端用交叉工具链、PC 端用本机编译器。分层检查在构建时执行，上层引用下层内部符号即报错。

2.3.2.4 素材与启动

图 15 展示从原图到板上可用的过程。PNG 图集与点阵字库经打包工具转为预乘 ARGB，素材包保存目录、尺寸与像素；启动后从 Flash 读包并校验，写入 DDR3、登记表面，再把热集复制到片上纹理。绘制时命令只引用表面与图集矩形，无需重复解码。



图 15　素材流水线与启动装载

第三部分　完成情况及性能参数

3.1 整体介绍

本作品的实物由 Ti60F225 开发板、HDMI 显示器及供电和输入连线构成。程序在板上的 RISC-V 软核执行，像素由 FPGA 引擎绘制后直接输出到显示器；上位机通过板载 USB 串口转发输入和采集日志。图 16 为系统实物：开发板经 HDMI 线连接显示器，屏上为应用 1 的方块演示，HUD 显示硬件渲染 60 FPS、活动元素 100。

当前成果包含二维分桶与三维光栅两种配置。前者用于比较二维执行路径及数量极限，后者用于三维场景；两者使用同一开发板，功能组合和资源占用分别列示。



图 16　系统实物图

3.2 工程成果

3.2.1 电路成果

硬件交付包括可执行命令表的绘图引擎、DDR 帧缓冲到 HDMI 的扫描链路，以及对应的软件控制接口。二维构型保留直通、片上纹理、条带和硬件分桶，可切换执行方法；三维构型保留片上纹理与条带，配置三维光栅和深度处理。表 3 给出整机编译结果，统计对象包含软核、存储接口、显示和引擎，所列建立与保持余量均为正。

表 3　两种整机构型的资源和时序

资源	二维构型	三维构型
XLR	60,064（98.8%）	59,623（98.06%）
片上 RAM	172（67.2%）	187（73.0%）
DSP	23（14.4%）	26（16.3%）
内核时钟 建立 / 保持	+0.104 / +0.022 ns	+0.762 / +0.026 ns

XLR 表示逻辑资源，片上 RAM 以块计，DSP 用于乘法等运算；其中 12 个 DSP 对应基础像素通路的 4 像素×RGB 三通道，其余承担相应构型的地址计算或光栅工作。两种构型均接近逻辑容量上限，DSP 使用比例仍较低，资源取舍主要受整机逻辑与存储组织约束。

两种构型目前均由 JTAG 装载运行。Flash 应用写入、回读与断电自启动已在开发过程中单独验证，但尚未把两种构型都固化为上电自启动，演示按 JTAG 装载进行。

3.2.2 软件成果

软件交付包括板上 C 驱动、PC 与板上双目标应用、素材打包工具以及游戏机前端。驱动负责提交与呈现；前端以菜单组织四个应用，弹幕游戏提供移动、射击场景、炸弹和关卡变化，HUD 同屏显示当前后端与帧率。用户可在菜单与应用内切换场景，具备对应能力时再切换渲染路径。

四个应用分别是基础演示、弹幕游戏、压力测试与三维走廊轨道射击：弹幕游戏包含半透明弹幕、色键角色与首领战，压力测试调节发射密度并自动搜索活动 Sprite 极限。图 17 至图 19 为三个应用在 PC 上的界面，展示同一前端下的画面与交互；板上实际运行见图 16，三维性能见 3.3.4 节。

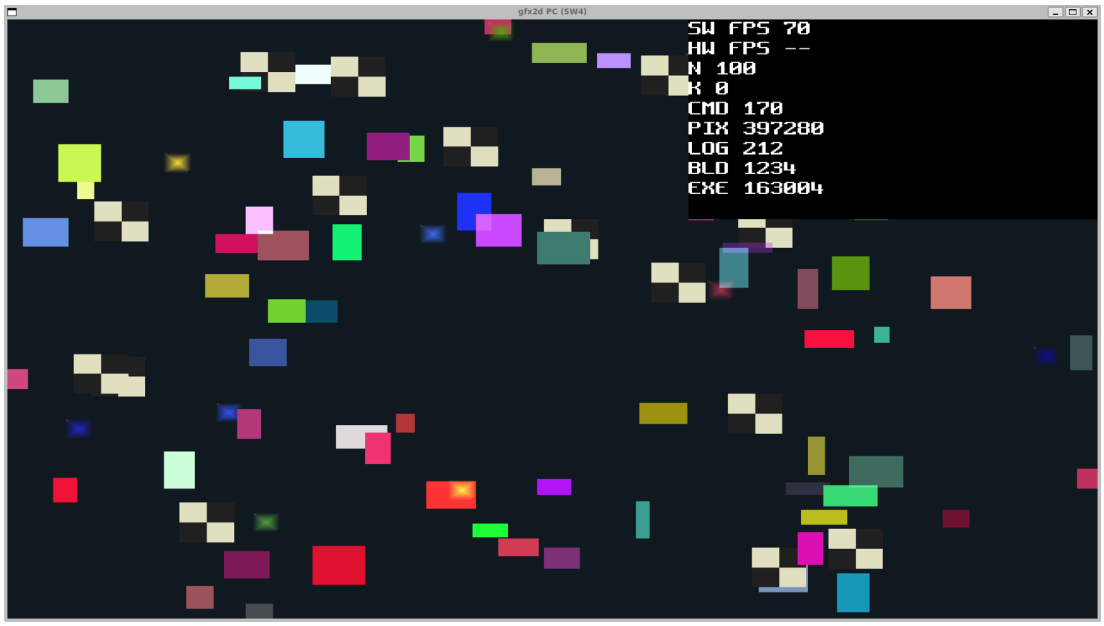


图 17　应用 1 基础演示

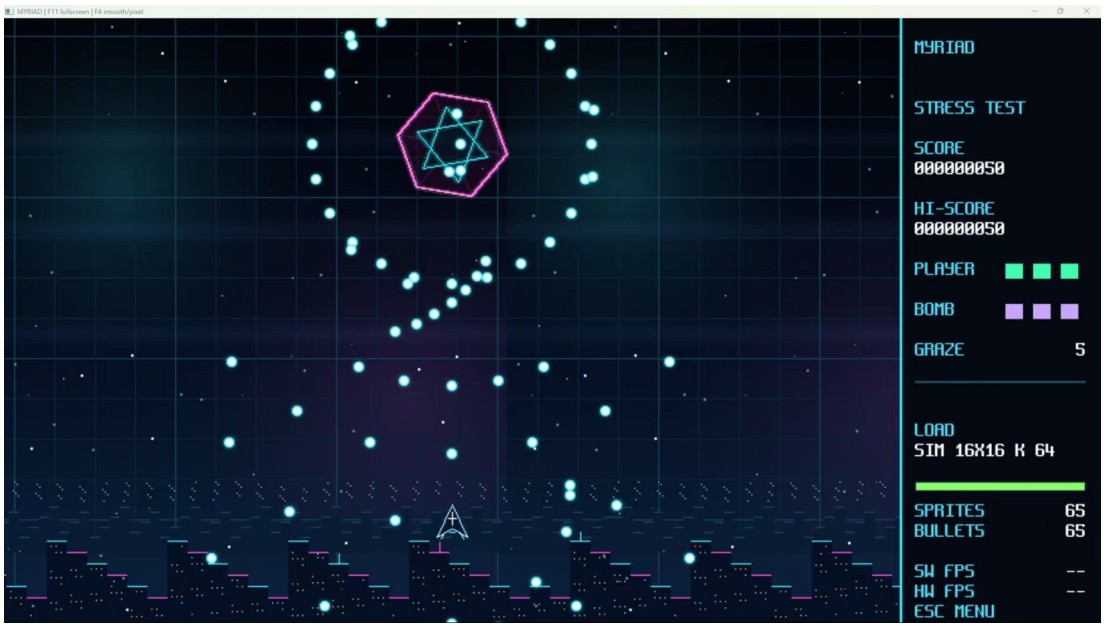


图 18　应用 2 弹幕游戏

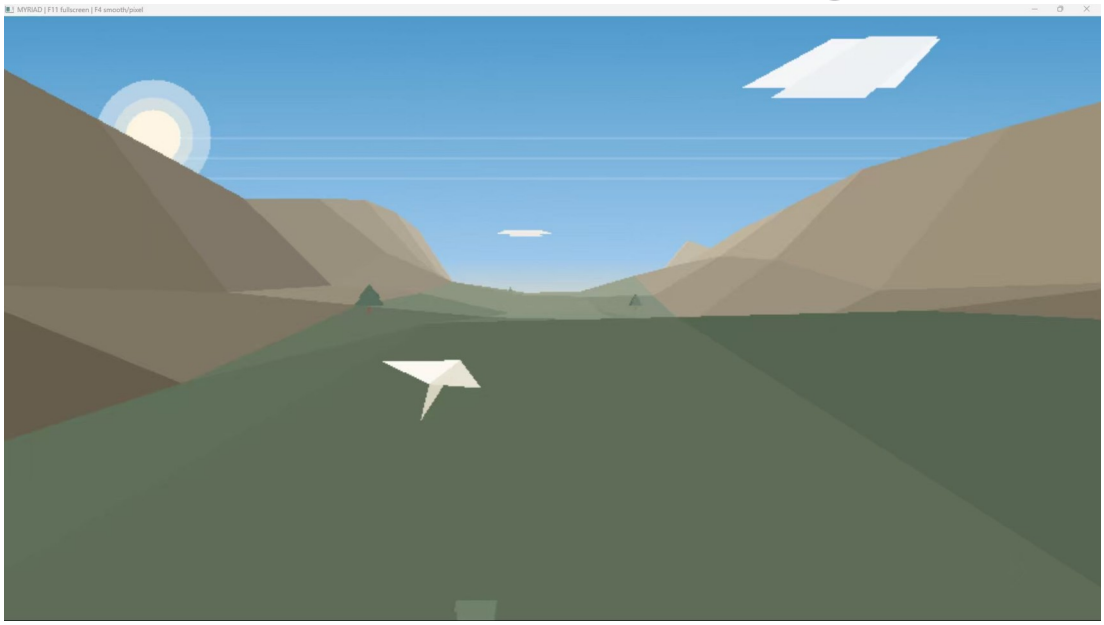


图 19　应用 3 三维场景

3.3 特性成果

本节按优化顺序报告结果：基础加速与负载极限、片上纹理与条带、硬件分桶，最后是三维扩展。N_{max} 表示指定尺寸 Sprite 在真实游戏中保持稳定 60 FPS 的活动数量极限；K 是负载旋钮，N_{max} 是实际存量，成对结果依次对应 16×16 与 8×8。

3.3.1 基础加速与负载极限

基础直通版本使用相同游戏与绘制语义，在 CPU 软件执行和 FPGA 硬件执行之间切换。普通游戏短测中，硬件首秒预热后达到 60 FPS，纯软件约 2.5 FPS，约加速 24 倍。CPU 提交绘图任务后不再逐像素搬运和混合，因而能够维持实时游戏显示，图 20 左侧给出这一对比。

压力档约有 2030 颗活动弹，硬件 60 秒呈现 1839 帧，约 30.6 FPS，说明更高密度仍会超过单场预算。软件路径此时无法维持同一更新节拍，场上活动弹存量下降，图中右侧用于展示过载行为，不据此计算同规模加速比。统计结论采用场数与交换数。

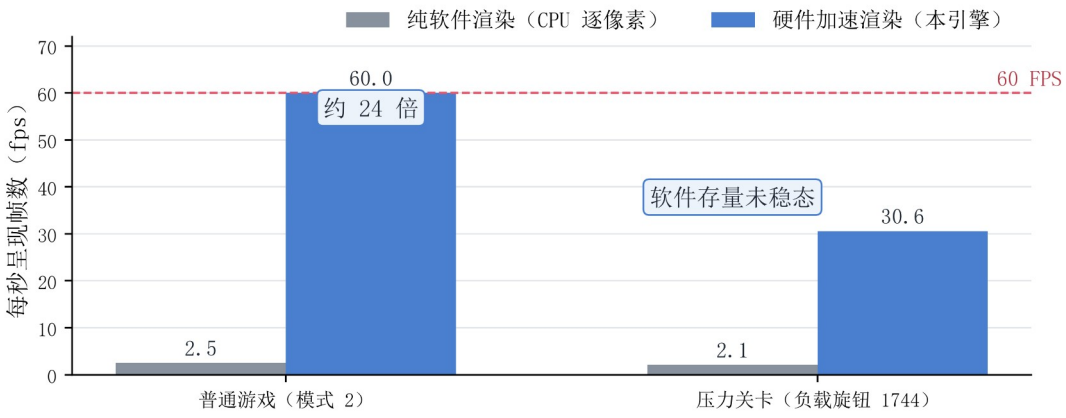


图 20　普通游戏与过载表现

基础直通版本的 N_{max} 为 1014（16×16，K896）和 1365（8×8，K1216），各完成 3600 场稳定确认，搜索轨迹见图 21。较小 Sprite 的像素面积减少，数量极限随之提高，但没有按面积同比例增长，说明每对象的取指、访存和调度等成本也影响整机能力；这一现象本身不能单独定位 CPU 或 DDR 瓶颈。

独立 K896 固定负载运行中，帧时间约为 16.3 ms，首秒建立流水，其后 59 个逐秒窗口均为 60/60，见图 22。该结果描述基础版本的持续负载能力。后续构型改变了访存和命令路径，采用各自同条件对照，不直接用历史成绩差值归因。

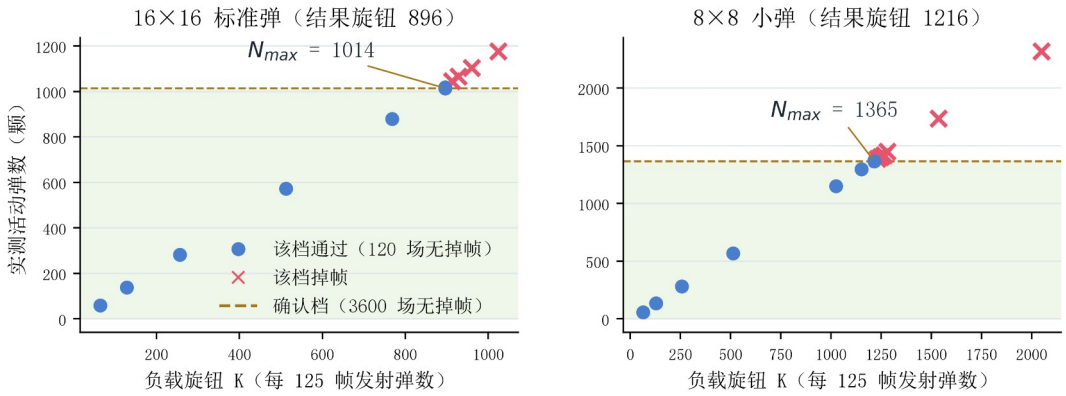


图 21　N_{max} 搜索轨迹

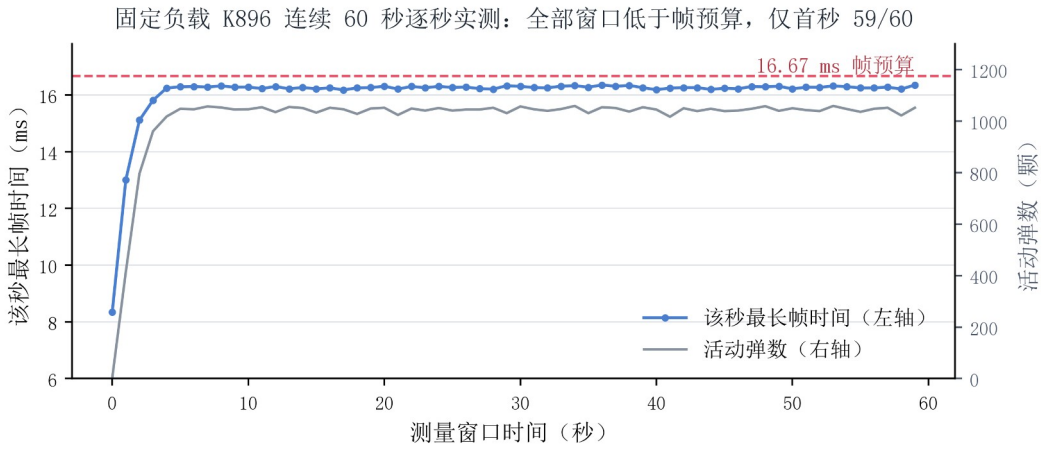


图 22　固定负载连续运行

3.3.2 片上纹理与条带对照

片上纹理让圆弹等热素材从片上反复读取，直通路径仍按对象顺序访问目的帧缓冲；条带路径进一步把同一区域的合成留在片上。表 4 在同一版本内比较直通片上纹理与条带两种执行，并在条带程序内比较 CPU 数据缓存的容量与路数。

4 KB 缓存下，直通片上纹理的 N_{max} 为 1891 / 2237，条带为 424 / 505。条带虽然减少了目的数据反复进出 DDR，CPU 还要为每条带求交、调整坐标并复制命令。分段测得 CPU 逻辑与组帧约需 13 ms、引擎约需 6 ms，CPU 侧成为主要成本。两者是局部分段，提交和同步等仍占帧预算，最终是否达标以实际呈现为准。

表 4　片上纹理、条带及缓存容量对照

构型与后端	16×16 N_{max} / K	8×8 N_{max} / K
4 KB，直通片上纹理	1891 / 1648	2237 / 1984
4 KB，条带	424 / 384	505 / 464
16 KB 候选，条带	479 / 432	597 / 544

将 CPU 数据缓存由 4 KB / 1 路改为 16 KB / 4 路候选后，条带 N_{max} 提高约 13% / 18%。这是容量和路数共同变化的构型差距，仍未消除组帧成本。该

阶段的结果使优化目标从目的写回转向命令构造：保留条带的局部合成方式，把 CPU 反复进行的分桶与复制交给硬件。

3.3.3 硬件分桶的同程序收益

二维构型将分桶计算和保序引用构建移入引擎，CPU 发布原始绘制与实例数组，随后由硬件组织各条带任务。对照程序同时携带直通片上纹理与硬件分桶两条路径，在同一比特流、素材及负载定义下运行时切换，以减少程序版本差异。

表 5 采用开发定稿记录：16×16 的 N_{max} 由 1872 提升到 2189，8×8 由 1947 提升到 2187，分别提高 16.9% 和 12.3%，各完成 3600 场稳定确认。该结果表明，减少 CPU 任务构造后，条带组织能够在这组真实负载中获得整机收益。

表 5　同一比特流、同一程序的后端对照（开发定稿记录）

后端	16×16 N_{max} / K	8×8 N_{max} / K
直通片上纹理	1872 / 1632	1947 / 1728
硬件分桶	2189 / 1904	2187 / 1936

总线流量另取同一程序的独立复现记录，在各自确认档上比较。两路 16×16 极限档中，分桶承载更多活动弹，每帧读突发均值仍由直通约 32,493 次减至 9863 次，写突发由约 39,835 次减至 3286 次，分别减少约 70% 和 92%。引用表比跨桶复制的完整命令紧凑，任务取用与目的合成共同降低事务数量；这些流量属于各自确认档，不是固定活动数量的独立带宽测试。

按 N_{max}×Sprite 面积×60 换算，分桶两档的等效混合吞吐约为 33.62 / 8.40 Mpix/s，包含游戏状态、调度与访存条件。像素阵列理论 400 Mpix/s 则假设流水持续供数，两类指标描述不同层次，分别报告。

图 23 汇集各阶段的绝对成绩，用于观察不同组织方式的负载能力。表 4 的直通 2237 与表 5 的直通 1947 属于不同程序，后者还携带分桶发布和缓冲

管理路径。因此，图中历史柱形不能直接连接成单项优化的增长率，硬件分桶收益以表 5 的同程序结果为准。

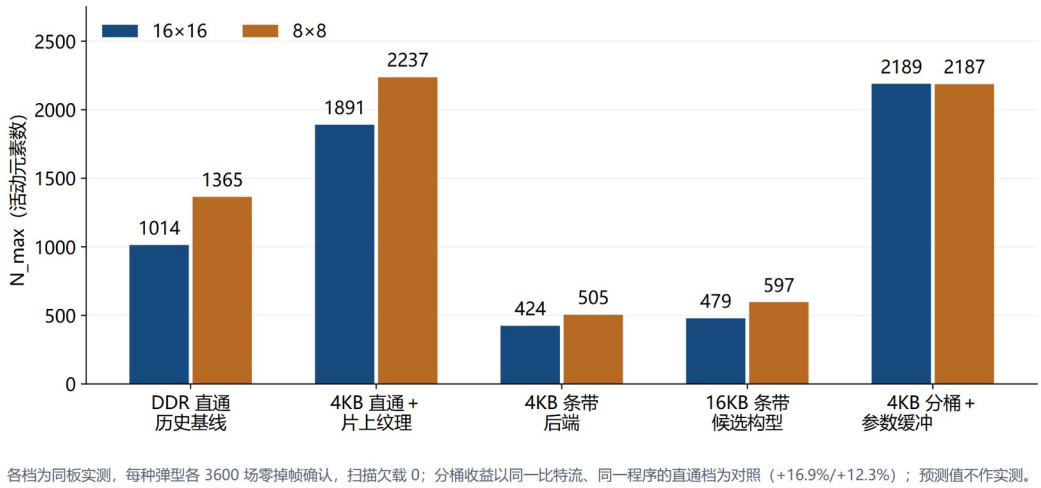


图 23　N_{max} 汇总

3.3.4 三维渲染表现

三维应用沿用二维的条带执行方式，把处理对象换成三角形：三维走廊轨道射击透视绘制门框、敌机与弹幕，支持交互移动与炸弹输入，敌机与门框由远及近穿过视野，深度通路处理前后遮挡，二维通路叠加 HUD。

走廊场景以自动驾驶与脚本输入两种方式各连续测量 1800 帧，均记录 3600 场与 1800 次交换，每两场呈现一帧。表 6 给出：自动驾驶 p50 25.96 ms、最大 30.96 ms，脚本输入 p50 27.26 ms、最大 32.34 ms，两组最大值均低于 33.3 ms 预算、迟到 0。

表 6　三维走廊轨道射击的连续帧时间

输入	p50 / ms	p99 / ms	最大 / ms
自动驾驶	25.96	30.15	30.96
脚本移动与炸弹	27.26	31.30	32.34

三维构型的二维直通片上纹理 N_{max} 另测得 1870 / 1969，说明同一构型仍保留二维应用能力；该组未启用硬件分桶，成绩与二维构型分属不同构型。三维扩展复用条带调度、纹理与搬运基础，所需逻辑、RAM 与 DSP 见表 3。

第四部分　总结

4.1 可扩展之处

二维侧可进一步研究双条带缓冲，使一块缓冲执行时，另一块装载或写回，以减少阶段间空等；CPU 状态更新和提交管理仍需结合完整帧分析。三维侧已具备几何准备与光栅执行的分工，增加场景复杂度时需同时控制三角形数量、覆盖范围和 CPU setup 成本。

提高分辨率还需调整显示时序、帧缓冲布局与条带尺寸，并重新核算带宽和帧预算。当前两种构型接近逻辑容量上限，新增功能宜按目标负载选择组合；16 KB 缓存等候选的采用，也应以实际应用的整体收益和资源代价决定。

4.2 心得体会

本作品的开发使软硬件分工逐步具体化。最初把像素搬运和混合交给硬件，即可明显改善软件逐像素路径的帧率；面对更密集的真实画面，还必须处理素材复用、目的区域重叠和任务准备。像素流水线的并行度只描述计算能力，整机性能取决于数据和任务能否持续送到这条流水线。

条带模式提供了直接经验：集中目的访问能够减少引擎侧工作，却会让 CPU 承担分桶、坐标调整和命令复制。该阶段的整机成绩下降，使后续设计把注意力放到命令组织；硬件分桶保留原有像素语义与绘制顺序，将可重复的构造工作下沉，才在同条件比较中得到整体收益。

真实游戏也让代价更完整。背景、HUD、输入、状态更新和呈现都要占用一帧，单个模块的速度无法直接代替最终画面能力。保留直通和条带等执行路径，有助于针对负载选择实现；二维分桶与三维分别配置，则是在有限逻辑和存储资源中围绕目标功能进行取舍。

第五部分　参考文献

[1] 全国大学生嵌入式芯片与系统设计竞赛组委会. 2026 FPGA 创新设计赛道选题指南：易灵思[R]. 2026.

[2] EFINIX INC. Sapphire RV32 SoC User Guide: UG-RISCV-SAPPHIRE-v8.2[R]. 2026.

[3] 易灵思. Ti60F225I3-V4 DEMO 板软硬件设计说明[R]. 2026.

[4] PORTER T, DUFF T. Compositing digital images[J]. ACM SIGGRAPH Computer Graphics, 1984, 18(3): 253-259.

[5] AKENINE-MÖLLER T, HAINES E, HOFFMAN N, et al. Real-Time Rendering[M]. 4th ed. Boca Raton: CRC Press, 2018.

[6] PINEDA J. A parallel algorithm for polygon rasterization[J]. ACM SIGGRAPH Computer Graphics, 1988, 22(4): 17-20.

[7] EFINIX INC. DDR3 Soft Controller Core User Guide: UG-CORE-DDR3-CONTROLLER-v2.1[R]. 2024.

[8] ARM LIMITED. AMBA AXI and ACE Protocol Specification: ARM IHI 0022H[S]. 2020.

[9] Wikimedia Commons. Vital signs monitor display（美国海军 James Stenberg 摄，公有领域）；Digital instrument cluster of a Toyota Yaris Hybrid GR（Acgskup 摄，CC BY 4.0）[EB/OL]. https://commons.wikimedia.org

AI 工具使用说明

本作品使用 AI 辅助资料检索、代码实现与审查、文稿校对和图表生成。文中功能与性能依据本机工具及板卡的实际运行记录，引用数字由数据台账追溯；AI 辅助生成的文字与代码由项目开发过程核对。